Homework #3

CSE 546: Machine Learning
Tyler Shakibai

Collaborators: Jake Murray, Peter Sushko

Conceptual Questions

A1. The answers to these questions should be answerable without referring to external materials. Briefly justify your answers with a few words.

a. *[2 points]* Say you trained an SVM classifier with an RBF kernel ($K(u, v) = \exp\left(-\frac{\|u-v\|_2^2}{2\sigma^2}\right)$). It seems to underfit the training set: should you increase or decrease σ?

You should decrease σ. Given $\gamma = \frac{1}{2\sigma^2}$, decreasing γ increases the margin while increasing γ tightens the margin. We want to decrease the margin and thus should decrease σ.

b. *[2 points]* True or False: Training deep neural networks requires minimizing a convex loss function, and therefore gradient descent will provide the best result.

False. The loss function is non-convex, and therefore gradient descent might not reach the optimal solution.

c. *[2 points]* True or False: It is a good practice to initialize all weights to zero when training a deep neural network.

False. It is not a good practice to initialize all weights to the same value when training a deep neural network because the neurons would learn the same features during training.

d. *[2 points]* True or False: We use non-linear activation functions in a neural network's hidden layers so that the network learns non-linear decision boundaries.

True. Non-linear activation functions introduce non-linearity into the hidden layers. Without these non-linear activation functions, the network will be a linear combination of inputs.

e. *[2 points]* True or False: Given a neural network, the time complexity of the backward pass step in the backpropagation algorithm can be prohibitively larger compared to the relatively low time complexity of the forward pass step.

False. While the complexity of the backward pass step is larger, both the forward and backward passes are of time complexity $O(n)$ and thus it is not prohibitively larger.

f. *[2 points]* True or False: Neural Networks are the most extensible model and therefore the best choice for any circumstance.

False. Neural networks are not always the best choice because they require a lot of data. If there is less data available, other learning algorithms may perform better.

Support Vector Machines

A2. Recall that solving the SVM problem amounts to solving the following constrained optimization problem:

$$\text{Given data points } \mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n} \text{ find}$$

$$\min_{w,b} ||w||_2 \text{ subject to } y_i(x_i^T w - b) \geq 1 \text{ for } i \in \{1, \ldots, n\}$$

$$\text{where } x_i \in \mathbb{R}^d,\ y_i \in \{-1, 1\}, \text{ and } w \in \mathbb{R}^d.$$

Consider the following labeled data points:

$$\begin{bmatrix} 1 & 2 \\ 1 & 3 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \text{ with label } y = -1 \text{ and } \begin{bmatrix} 0 & 0.5 \\ 1 & 0 \\ 2 & 1 \\ 3 & 0 \end{bmatrix} \text{ with label } y = 1$$

a. *[2 points]* Graph the data points above. Highlight the support vectors and write their coordinates. Draw the two parallel hyperplanes separating the two classes of data such that the distance between them is as large as possible. Draw the maximum-margin hyperplane. Write the equations describing these three hyperplanes using only x, w, b(that is without using any specific values). Draw w(it doesn't have to have the exact magnitude, but it should have the correct orientation).

The support vectors are $(x_1, x_2) = (0, 0.5), (2, 1), (1, 2)$.

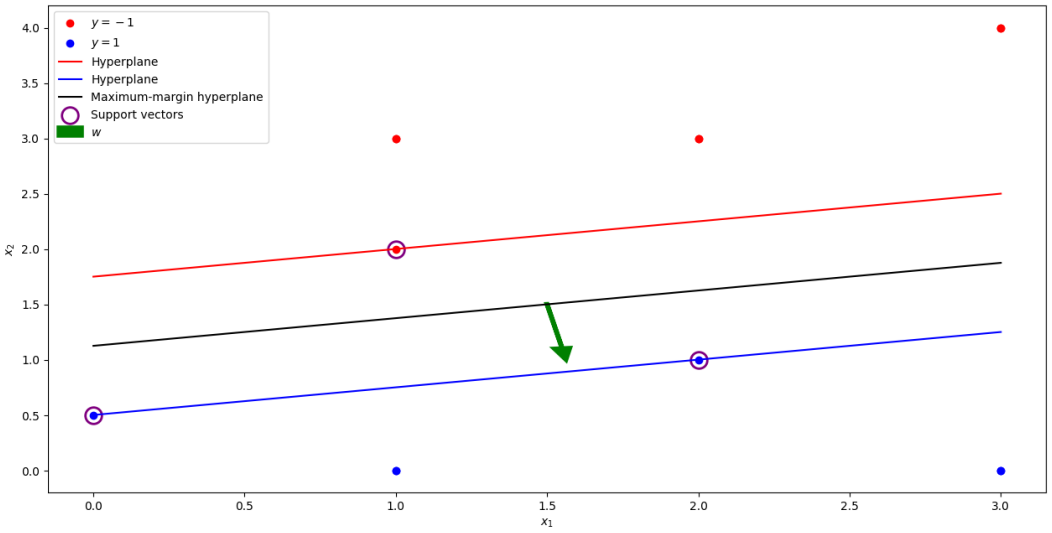


Figure 1: Plot of the data points with the support vectors circled in purple, the two parallel hyperplanes are the red and blue lines, the maximum-margin hyperplane is the black line, and the green arrow denotes w.

The equations describing these hyperplanes are

$$0w_1 + 0.5w_2 + b = 1$$

$$2w_1 + 1w_2 + b = 1$$

$$1w_1 + 2w_2 + b = -1$$

b. *[2 points]* For the data points above, find w and b.

Hint: Use the support vectors and the values $\{-1, 1\}$ to create a linear system of equations where the unknowns are w_1, w_2 and b.

$$0w_1 + 0.5w_2 + b = 1$$
$$\Rightarrow w_2 = -2b + 2$$
$$1w_1 + 2w_2 + b = -1$$
$$\Rightarrow w_1 = -2w_2 - b - 1$$
$$2w_1 + 1w_2 + b = 1$$

Substituting

$$2(-2w_2 - b - 1) + w_2 + b = 1$$
$$\Rightarrow 3w_2 + b = -3$$
$$3(-2b + 2) + b = -3$$
$$\Rightarrow -5b + 6 = -3$$
$$\Rightarrow b = 9/5$$
$$0.5w_2 + 9/5 = 1$$
$$\Rightarrow 0.5w_2 = -4/5$$
$$\Rightarrow w_2 = -8/5$$
$$w_1 + 2w_2 + b = -1$$
$$\Rightarrow w_1 = -2(\frac{-8}{5}) - 9/5 - 1$$
$$\Rightarrow w_1 = \frac{2}{5}$$
$$\therefore w_1 = \frac{2}{5}, w_2 = -8/5, b = 9/5$$

c. *[4 points]* Show that for any solvable SVM problem, the distance between the two separating hyperplanes is $\frac{2}{||w||_2}$.

Hint 1: The distance between two hyperplanes is the distance between any point x_0 on one of the hyperplanes and its projection on the other hyperplane.
Hint 2: A direction w and an offset c define the hyperplane: $H = \{x \in \mathbb{R}^n | w^T x = c\}$. The projection of a vector y onto H is given by $P_H(y) = y - \frac{w^T y - c}{||w||_2^2} w$.

Let x_0 be a point on the hyperplane $H_0 = \{x | w^T x = -1\}$ and x_1 be a point on the hyperplane $H_1 = \{x | w^T x = 1\}$. The distance between the two hyperplanes is equal to the magnitude of the projection vector $\overrightarrow{x_0 x_1}$ onto the unit vector w. Hence

$$\text{distance} = \left| (\vec{x_1}) - (\vec{x_0}) \cdot \frac{w}{|w|} \right| = \left| \frac{w \cdot \vec{x_1} - w \cdot \vec{x_0}}{||w||_2} \right| = \left| \frac{(1 - b) - (b - 1)}{||w||_2} \right| = \left| \frac{2}{||w||_2} \right| = \frac{2}{||w||_2}$$

Kernels

A3. *[5 points]* Suppose that our inputs x are one-dimensional and that our feature map is infinite-dimensional: $\phi(x)$ is a vector whose ith component is:

$$\frac{1}{\sqrt{i!}} e^{-x^2/2} x^i \ ,$$

for all nonnegative integers i. (Thus, ϕ is an infinite-dimensional vector.) Show that $K(x, x') = e^{-\frac{(x-x')^2}{2}}$ is a kernel function for this feature map, i.e.,

$$\phi(x) \cdot \phi(x') = e^{-\frac{(x-x')^2}{2}} \ .$$

Hint: Use the Taylor expansion of $z \mapsto e^z$. (This is the one dimensional version of the Gaussian (RBF) kernel).

$$\phi(x)\phi(x') = \sum_{i=1}^{\infty} \left(\frac{1}{\sqrt{i!}} e^{-x^2/2} x^i \right) \left(\frac{1}{\sqrt{i!}} e^{-x'^2/2} x'^i \right) = \sum_{i=1}^{\infty} \frac{1}{i!} x^i x'^i e^{\frac{-x^2-x'^2}{2}} = e^{\frac{-x^2-x'^2}{2}} \sum_{i=1}^{\infty} \frac{(xx')^i}{i!}$$

Using $e^x = \sum_{i=1}^{\infty} \frac{x^n}{n}$:

$$e^{\frac{-x^2-x'^2}{2}} \sum_{i=1}^{\infty} \frac{(xx')^i}{i!} = e^{\frac{-x^2-x'^2}{2}} e^{-xx'} = e^{-\frac{(x-x')^2}{2}}$$

A4. This problem will get you familiar with kernel ridge regression using the polynomial and RBF kernels. First, let's generate some data. Let $n = 30$ and $f_*(x) = 4\sin(\pi x)\cos(6\pi x^2)$. For $i = 1, \ldots, n$ let each x_i be drawn uniformly at random from $[0, 1]$, and let $y_i = f_*(x_i) + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, 1)$. For any function f, the true error and the train error are respectively defined as:

$$\mathcal{E}_{\text{true}}(f) = \mathbb{E}_{X,Y}\left[(f(X) - Y)^2\right], \qquad \widehat{\mathcal{E}}_{\text{train}}(f) = \frac{1}{n}\sum_{i=1}^{n} (f(x_i) - y_i)^2 \, .$$

Now, our goal is, using kernel ridge regression, to construct a predictor:

$$\widehat{\alpha} = \arg\min_{\alpha} \|K\alpha - y\|_2^2 + \lambda \alpha^\top K \alpha \ , \qquad \widehat{f}(x) = \sum_{i=1}^{n} \widehat{\alpha}_i k(x_i, x)$$

where $K \in \mathbb{R}^{n \times n}$ is the kernel matrix such that $K_{i,j} = k(x_i, x_j)$, and $\lambda \geq 0$ is the regularization constant.

a. *[10 points]* Using leave-one-out cross validation, find a good λ and hyperparameter settings for the following kernels:

- $k_{\text{poly}}(x, z) = (1 + x^\top z)^d$ where $d \in \mathbb{N}$ is a hyperparameter,
- $k_{\text{rbf}}(x, z) = \exp(-\gamma \|x - z\|_2^2)$ where $\gamma > 0$ is a hyperparameter[1].

We strongly recommend implementing either grid search or random search. **Do not use sklearn**, but actually implement of these algorithms. Reasonable values to look through in this problem are: $\lambda \in 10^{[-5,-1]}$, $d \in [5, 25]$, γ sampled from a narrow gaussian distribution centered at value described in the footnote. Report the values of d, γ, and the λ values for both kernels.

Poly kernel: $\lambda = 2.310e-05, \quad d = 19$
RBF kernel: $\lambda = 1e-05, \quad \gamma = 13.290$

b. *[10 points]* Let $\widehat{f}_{\text{poly}}(x)$ and $\widehat{f}_{\text{rbf}}(x)$ be the functions learned using the hyperparameters you found in part a. For a single plot per function $\widehat{f} \in \left\{ \widehat{f}_{\text{poly}}(x), \widehat{f}_{\text{rbf}}(x) \right\}$, plot the original data $\{(x_i, y_i)\}_{i=1}^n$, the true $f(x)$, and $\widehat{f}(x)$ (i.e., define a fine grid on $[0, 1]$ to plot the functions).

[1] Given a dataset $x_1, \ldots, x_n \in \mathbb{R}^d$, a heuristic for choosing a range of γ in the right ballpark is the inverse of the median of all $\binom{n}{2}$ squared distances $\|x_i - x_j\|_2^2$.

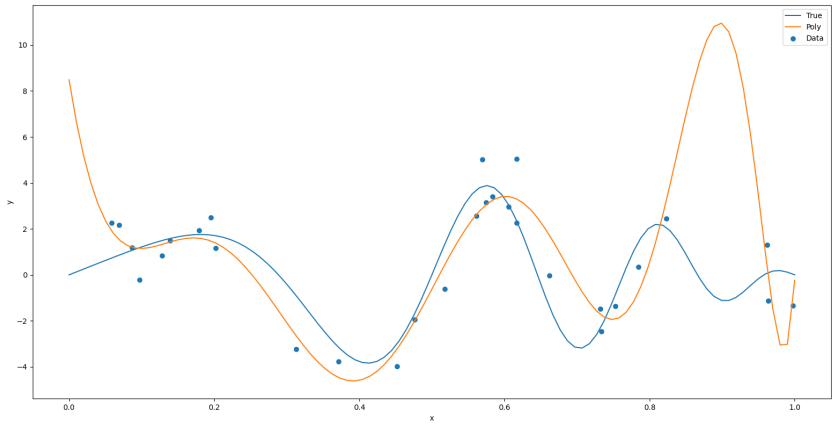


Figure 2: Poly kernel plotted against true $f(x)$.

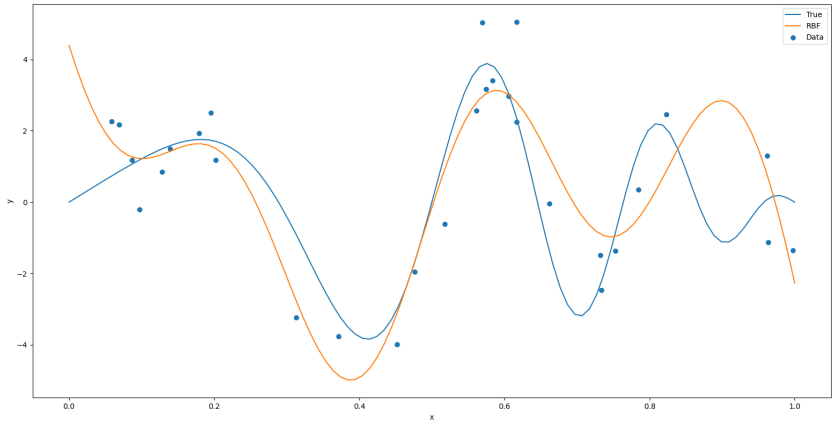


Figure 3: RBF kernel plotted against true $f(x)$.

```
def f_true(x: np.ndarray) -> np.ndarray:
    """True function, which was used to generate data.
    Should be used for plotting.

    Args:
        x (np.ndarray): A (n,) array. Input.

    Returns:
        np.ndarray: A (n,) array.
    """
    return 4 * np.sin(np.pi * x) * np.cos(6 * np.pi * x ** 2)


@problem.tag("hw3-A")
def poly_kernel(x_i: np.ndarray, x_j: np.ndarray, d: int) -> np.ndarray:
    """Polynomial kernel.
```

```
    Given two indices a and b it should calculate:
    K[a, b] = (x_i[a] * x_j[b] + 1)^d

    Args:
        x_i (np.ndarray): An (n,) array. Observations (Might be different from x_j).
        x_j (np.ndarray): An (m,) array. Observations (Might be different from x_i).
        d (int): Degree of polynomial.

    Returns:
        np.ndarray: A (n, m) matrix, where each element is as described above (see equa

    Note:
        - It is crucial for this function to be vectorized, and not contain for-loops.
            It will be called a lot, and it has to be fast for reasonable run-time.
        - You might find .outer functions useful for this function.
            They apply an operation similar to xx^T (if x is a vector), but not necessa
            To use it simply append .outer to function. For example: np.add.outer, np.d
    """
    return (np.outer(x_i, x_j) + 1)**d


@problem.tag("hw3-A")
def rbf_kernel(x_i: np.ndarray, x_j: np.ndarray, gamma: float) -> np.ndarray:
    """Radial Basis Function (RBF) kernel.

    Given two indices a and b it should calculate:
    K[a, b] = exp(-gamma*(x_i[a] - x_j[b])^2)

    Args:
        x_i (np.ndarray): An (n,) array. Observations (Might be different from x_j).
        x_j (np.ndarray): An (m,) array. Observations (Might be different from x_i).
        gamma (float): Gamma parameter for RBF kernel. (Inverse of standard deviation)

    Returns:
        np.ndarray: A (n, m) matrix, where each element is as described above (see equa

    Note:
        - It is crucial for this function to be vectorized, and not contain for-loops.
            It will be called a lot, and it has to be fast for reasonable run-time.
        - You might find .outer functions useful for this function.
            They apply an operation similar to xx^T (if x is a vector), but not necessa
            To use it simply append .outer to function. For example: np.add.outer, np.d
    """
    return np.e**(-gamma * np.subtract.outer(x_i, x_j)**2)


@problem.tag("hw3-A")
def train(
    x: np.ndarray,
    y: np.ndarray,
    kernel_function: Union[poly_kernel, rbf_kernel],  # type: ignore
    kernel_param: Union[int, float],
    _lambda: float,
) -> np.ndarray:
```

```
    """Trains and returns an alpha vector, that can be used to make predictions.

    Args:
        x (np.ndarray): Array of shape (n,). Observations.
        y (np.ndarray): Array of shape (n,). Targets.
        kernel_function (Union[poly_kernel, rbf_kernel]): Either poly_kernel or rbf_kern
        kernel_param (Union[int, float]): Gamma (if kernel_function is rbf_kernel) or d
        _lambda (float): Regularization constant.

    Returns:
        np.ndarray: Array of shape (n,) containing alpha hat as described in the pdf.
    """
    return np.linalg.solve(kernel_function(x, x, kernel_param) + _lambda * np.eye(len(k


@problem.tag("hw3-A", start_line=1)
def cross_validation(
    x: np.ndarray,
    y: np.ndarray,
    kernel_function: Union[poly_kernel, rbf_kernel],  # type: ignore
    kernel_param: Union[int, float],
    _lambda: float,
    num_folds: int,
) -> float:
    """Performs cross validation.

    In a for loop over folds:
        1. Set current fold to be validation, and set all other folds as training set.
        2, Train a function on training set, and then get mean squared error on current
    Return validation loss averaged over all folds.

    Args:
        x (np.ndarray): Array of shape (n,). Observations.
        y (np.ndarray): Array of shape (n,). Targets.
        kernel_function (Union[poly_kernel, rbf_kernel]): Either poly_kernel or rbf_kern
        kernel_param (Union[int, float]): Gamma (if kernel_function is rbf_kernel) or d
        _lambda (float): Regularization constant.
        num_folds (int): Number of folds. It should be either len(x) for LOO, or 10 for

    Returns:
        float: Average loss of trained function on validation sets across folds.
    """
    fold_size = len(x) // num_folds
    err = []
    for fold in range(num_folds):
        currStart, currEnd = fold * fold_size, (fold + 1) * fold_size
        x_train, y_train = np.append(x[:currStart], x[currEnd:]), np.append(y[:currStart
        x_test, y_test = x[currStart: currEnd], y[currStart: currEnd]

        pred = train(x_train, y_train, kernel_function, kernel_param, _lambda) @ kernel_
        err.append(np.mean((pred - y_test)**2))

    return np.mean(err)
```

```
@problem.tag("hw3-A")
def rbf_param_search(
    x: np.ndarray, y: np.ndarray, num_folds: int
) -> Tuple[float, float]:
    """
    Parameter search for RBF kernel.

    There are two possible approaches:
        - Grid Search - Fix possible values for lambda, loop over them and record value
        - Random Search - Fix number of iterations, during each iteration sample lambda

    Args:
        x (np.ndarray): Array of shape (n,). Observations.
        y (np.ndarray): Array of shape (n,). Targets.
        num_folds (int): Number of folds. It should be len(x) for LOO.

    Returns:
        Tuple[float, float]: Tuple containing best performing lambda and gamma pair.

    Note:
        - You do not really need to search over gamma. 1 / (median(dist(x_i, x_j)^2) for
          should be sufficient for this problem. That being said you are more than wel
        - If using random search we recommend sampling lambda from distribution 10**i, i
        - If using grid search we recommend choosing possible lambdas to 10**i, where i=
    """
    dist = []

    for i in range(len(x)):
        for j in range(i + 1, len(x)):
            dist.append((x[i] - x[j])**2)

    lambdas = 10 ** np.linspace(-5, -1, 100)
    gammas = np.random.normal(1 / np.median(dist), 1, 50)
    minErr = float("inf")
    minLam = None
    minGam = None

    for lam in lambdas:
        for gam in gammas:
            err = cross_validation(x, y, rbf_kernel, gam, lam, num_folds)
            if minErr > err:
                minGam = gam
                minErr = err
                minLam = lam

    return (minLam, minGam)

@problem.tag("hw3-A")
def poly_param_search(
    x: np.ndarray, y: np.ndarray, num_folds: int
) -> Tuple[float, int]:
    """
    Parameter search for Poly kernel.

    There are two possible approaches:
```

```
        - Grid Search - Fix possible values for lambdas and ds.
          Have nested loop over all possibilities and record value with the lowest los
        - Random Search - Fix number of iterations, during each iteration sample lambda,

    Args:
        x (np.ndarray): Array of shape (n,). Observations.
        y (np.ndarray): Array of shape (n,). Targets.
        num_folds (int): Number of folds. It should be either len(x) for LOO, or 10 for

    Returns:
        Tuple[float, int]: Tuple containing best performing lambda and d pair.

    Note:
        - You can use gamma = 1 / median((x_i - x_j)^2) for all unique pairs x_i, x_j in
          However, if you would like to search over other possible values of gamma, you
        - If using random search we recommend sampling lambda from distribution 10**i, i
          and d from distribution [5, 6, ..., 24, 25]
        - If using grid search we recommend choosing possible lambdas to 10**i, where i=
          and possible ds to [5, 6, ..., 24, 25]
    """
    minErr = float("inf")
    minLam = None
    minDeg = None

    lambdas = 10 ** np.linspace(-5, -1, 100)
    degrees = np.arange(5, 26)

    for lam in lambdas:
        for deg in degrees:
            err = cross_validation(x, y, poly_kernel, deg, lam, num_folds)
            if minErr > err:
                minDeg = deg
                minErr = err
                minLam = lam

    return (minLam, minDeg)

@problem.tag("hw3-A", start_line=1)
def main():
    """
    Main function of the problem

    It should:
        A. Using x_30, y_30, rbf_param_search and poly_param_search report optimal value
           Note that x_30, y_30 has been loaded in for you. You do not need to use (x_3
        B. For both rbf and poly kernels, train a function using x_30, y_30 and plot pr

    Note:
        - In part b fine grid can be defined as np.linspace(0, 1, num=100)
        - When plotting you might find that your predictions go into hundreds, causing
          To avoid this call plt.ylim(-6, 6).
    """
    (x_30, y_30), (x_300, y_300), (x_1000, y_1000) = load_dataset("kernel_bootstrap")

    x_span = np.linspace(0, 1, 100)
```

```
polyParams30 = poly_param_search(x_30, y_30, 10)
rbfParams30= rbf_param_search(x_30, y_30, len(x_30))

alpha_poly = train(x_30, y_30, poly_kernel, polyParams30[1], polyParams30[0])
alpha_rbf = train(x_30, y_30, rbf_kernel, rbfParams30[1], rbfParams30[0])

ypred_poly = alpha_poly.dot(poly_kernel(x_30, x_span, polyParams30[1]))
ypred_rbf = alpha_rbf.dot(rbf_kernel(x_30, x_span, rbfParams30 [1]))
ytrue = f_true(x_span)

print(polyParams30)
print(rbfParams30)

plt.scatter(x_30, y_30, label='Data')
plt.plot(x_span, ytrue, label='True')
plt.plot(x_span, ypred_poly, label='Poly')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()

plt.scatter(x_30, y_30, label='Data')
plt.plot(x_span, ytrue, label='True')
plt.plot(x_span, ypred_rbf, label='RBF')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

Perceptron

B1. One of the oldest algorithms used in machine learning (from the early 60's) is an online algorithm for learning a linear threshold function called the Perceptron Algorithm. It works as follows:

1. Start with the all-zeroes weight vector $\mathbf{w}_1 = 0$, and initialize t to 1. Also let's automatically scale all examples $\mathbf{x}$ to have (Euclidean) norm 1, since this doesn't affect which side of the plane they are on.
2. Given example $\mathbf{x}$, predict positive iff $\mathbf{w}_t \cdot \mathbf{x} > 0$.
3. On a mistake, update as follows:
   - Mistake on positive: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \mathbf{x}$.
   - Mistake on negative: $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \mathbf{x}$.
4. $t \leftarrow t + 1$.

If we make a mistake on a positive $\mathbf{x}$ we get $\mathbf{w}_{t+1} \cdot \mathbf{x} = (\mathbf{w}_t + \mathbf{x}) \cdot \mathbf{x} = \mathbf{w}_t \cdot \mathbf{x} + 1$, and similarly if we make a mistake on a negative $\mathbf{x}$ we have $\mathbf{w}_{t+1} \cdot \mathbf{x} = (\mathbf{w}_t - \mathbf{x}) \cdot \mathbf{x} = \mathbf{w}_t \cdot \mathbf{x} - 1$. So, in both cases we move closer (by 1) to the value we wanted. Here is a link if you are interested in more details.

Now consider the linear decision boundary for classification (labels in $\{-1, 1\}$) of the form $\mathbf{w} \cdot \mathbf{x} = 0$ (i.e., no offset). Now consider the following loss function evaluated at a data point $(\mathbf{x}, y)$ which is a variant on the hinge loss.

$$\ell((\mathbf{x}, y), \mathbf{w}) = \max\{0, -y(\mathbf{w} \cdot \mathbf{x})\}.$$

a. *[2 points]* Given a dataset of $(\mathbf{x}_i, y_i)$ pairs, write down a single step of subgradient descent with a step size of η if we are trying to minimize

$$\frac{1}{n}\sum_{i=1}^{n} \ell((\mathbf{x}_i, y_i), \mathbf{w})$$

for $\ell(\cdot)$ defined as above. That is, given a current iterate $\widetilde{\mathbf{w}}$ what is an expression for the next iterate?

In subgradient descent, the update step for minimizing the objective function $\frac{1}{n}\sum_{i=1}^{n} \ell((\mathbf{x}_i, y_i), \mathbf{w})$ with the hinge loss can be written as

$$w_{\text{next}} = w - \eta \nabla w \left(\frac{1}{n}\sum_{i=1}^{n} \ell((\mathbf{x}_i, y_i), \mathbf{w}) \right)$$

To compute the subgradient of the objective function with respect to w, we consider the subgradients of the individual hinge loss terms. Thus

$$w_{\text{next}} = w - \eta \left(\frac{1}{n}\sum_{i=1}^{n} \frac{\partial L(x_i, y_i)}{\partial w} \right)$$

where

$$\frac{\partial L(x_i, y_i)}{\partial w} = \begin{cases} -x_i y_i, & \text{if } 1 - y_i(w \cdot x_i) > 0 \\ 0, & \text{otherwise} \end{cases}$$

b. *[2 points]* Use what you derived to argue that the Perceptron can be viewed as implementing SGD applied to the loss function just described (for what value of η)?

In the Perceptron algorithm, the update step for the weight vector can be written as

$$w_{\text{next}} = w - \eta \cdot xy$$

Comparing this update step to the update step in subgradient descent derived above

$$w_{\text{next}} = w - \eta \left(\frac{1}{n}\sum_{i=1}^{n} \frac{\partial L(x_i, y_i)}{\partial w} \right)$$

We can see that the update step in the Perceptron algorithm is equivalent to the subgradient descent update step if we set $\eta = 1$. Therefore, the Perceptron algorithm can be viewed as implementing subgradient descent applied to the derived loss function with a step size of $\eta = 1$

c. *[1 point]* Suppose your data was drawn i.i.d. and that there exists a $\mathbf{w}^*$ that separates the two classes perfectly. Provide an explanation for why hinge loss is generally preferred over the loss given above.

Hinge loss is generally preferred because it is differentiable, is a convex function, and encourages the learning algorithm to maximize the margin between the decision boundary and the data points, even when perfect separation is possible.

Introduction to PyTorch

A5. PyTorch is a great tool for developing, deploying and researching neural networks and other gradient-based algorithms. In this problem we will explore how this package is built, and re-implement some of its core components. Firstly start by reading `README.md` file provided in `intro_pytorch` subfolder. A lot of problem statements will overlap between here, readme's and comments in functions.

a. *[10 points]* You will start by implementing components of our own PyTorch modules. You can find these in folders: `layers`, `losses` and `optimizers`. Almost each file there should contain at least one problem function, including exact directions for what to achieve in this problem. Lastly, you should implement functions in `train.py` file.

```python
from typing import Optional

import torch
from torch import nn

from utils import problem


class LinearLayer(nn.Module):
    @problem.tag("hw3-A", start_line=1)
    def __init__(
        self, dim_in: int, dim_out: int, generator: Optional[torch.Generator] =
    ):
        """Linear Layer, which performs calculation of: x @ weight + bias

        In constructor you should initialize weight and bias according to dimen
        You should use torch.randn function to initialize them by normal distrib

        Both weight and bias should be of torch's type float.
        Additionally, for optimizer to work properly you will want to wrap both

        Args:
            dim_in (int): Number of features in data input.
            dim_out (int): Number of features output data should have.
            generator (Optional[torch.Generator], optional): Generator to use w
                If defined it should be passed into torch.randn function.
                Defaults to None.

        Note:
            - YOU ARE NOT ALLOWED to use torch.nn.Linear (or it's functional co
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
        """
        super().__init__()
        self.weights = nn.Parameter(torch.randn(dim_in, dim_out, generator = ge
        self.bias = nn.Parameter(torch.randn(dim_out, generator = generator))

    @problem.tag("hw3-A")
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Actually perform multiplication x @ weight + bias

        Args:
            x (torch.Tensor): More specifically a torch.FloatTensor, with shape
                Input data.
```

```
        Returns:
            torch.Tensor: More specifically a torch.FloatTensor, with shape of
                Output data.
        """
        return x.float() @ self.weights + self.bias

import torch
from torch import nn

from utils import problem


class ReLULayer(nn.Module):
    @problem.tag("hw3-A")
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Performs a Rectified Linear Unit calculation (ReLU):
        Element-wise:
            - if x > 0: return x
            - else: return 0

        Args:
            x (torch.Tensor): More specifically a torch.FloatTensor, with some
                Input data.

        Returns:
            torch.Tensor: More specifically a torch.FloatTensor, with the same
                Every negative element should be substituted with 0.
                Output data.

        Note:
            - YOU ARE NOT ALLOWED to use torch.nn.ReLU (or it's functional coun
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
        """
        x[x < 0] = 0
        return x

import torch
from torch import nn

from utils import problem


class SigmoidLayer(nn.Module):
    @problem.tag("hw3-A")
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Performs a sigmoid calculation:
        Element-wise given x return 1 / (1 + e^(-x))

        Args:
            x (torch.Tensor): More specifically a torch.FloatTensor, with some
                Input data.

        Returns:
            torch.Tensor: More specifically a torch.FloatTensor, with the same
                Every negative element should be substituted with sigmoid of th
                Output data.
```

```
        Note:
            - YOU ARE NOT ALLOWED to use torch.nn.Sigmoid (or torch.nn.function
              YOU CAN however use other aliases of sigmoid function in PyTorc
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
        """
        return 1 / (1 + torch.exp(-x))

import torch
from torch import nn

from utils import problem


class SoftmaxLayer(nn.Module):
    @problem.tag("hw3-A")
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Performs a softmax calculation.
        Given a matrix x (n, d) on each element performs:

        softmax(x) = exp(x_ij) / sum_k=0^d exp(x_ik)

        i.e. it first takes an exponential of each element,
            and that normalizes rows so that their L-1 norm is equal to 1.

        Args:
            x (torch.Tensor): More specifically a torch.FloatTensor, with shape
                Input data.

        Returns:
            torch.Tensor: More specifically a torch.FloatTensor, also with shape
                Each row has L-1 norm of 1, and each element is in [0, 1] (i.e.
                Output data.

        Note:
            - For a numerically stable approach to softmax (needed for the probl
                first subtract max of x from data (no need for dim argument, to
                This causes exponent to not blow up, and arrives to exactly the
            - YOU ARE NOT ALLOWED to use torch.nn.Softmax (or it's functional c
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
        """
        return torch.exp(x - torch.max(x)) / torch.sum(torch.exp(x - torch.max(x

import torch
from torch import nn

from utils import problem


class CrossEntropyLossLayer(nn.Module):
    @problem.tag("hw3-A")
    def forward(self, y_pred: torch.Tensor, y_true: torch.Tensor) -> torch.Tens
        """Calculate Crossentropy loss based on (normalized) predictions and tru

        For a single example (x is vector of predictions, y is correct class):
```

```
        cross_entropy(x, y) = -log(x[y])

        Args:
            y_pred (torch.Tensor): More specifically a torch.FloatTensor, with s
                Predictions of classes. Each row is normalized so that L-1 norm
                Input data.
            y_true (torch.Tensor): More specifically a torch.LongTensor, with sh
                Each element is an integer in range [0, c).
                Input data.

        Returns:
            torch.Tensor: More specifically a SINGLE VALUE torch.FloatTensor (i
                Should be a mean over all examples.
                Result.

        Note:
            - YOU ARE NOT ALLOWED to use torch.nn.CrossEntropyLoss / torch.nn.N
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
            - Not that this is different from torch.nn.CrossEntropyLoss, as it
        """
        return torch.mean(-torch.log(y_pred[torch.arange(len(y_true)), y_true]))

import torch
from torch import nn

from utils import problem


class MSELossLayer(nn.Module):
    @problem.tag("hw3-A")
    def forward(self, y_pred: torch.Tensor, y_true: torch.Tensor) -> torch.Tenso
        """Calculate MSE between predictions and truth values.

        Args:
            y_pred (torch.Tensor): More specifically a torch.FloatTensor, with s
                Input data.
            y_true (torch.Tensor): More specifically a torch.FloatTensor, with
                Input data.

        Returns:
            torch.Tensor: More specifically a SINGLE VALUE torch.FloatTensor (i
                Result.

        Note:
            - YOU ARE NOT ALLOWED to use torch.nn.MSELoss (or it's functional c
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
        """
        return torch.mean((y_pred - y_true)**2)

import torch

from utils import problem


class SGDOptimizer(torch.optim.Optimizer):
    @problem.tag("hw3-A", start_line=1)
```

```
    def __init__(self, params, lr: float) -> None:
        """Constructor for Stochastic Gradient Descent (SGD) Optimizer.

        Provided code contains call to super class, which will initialize param
        This class will only update the parameters provided to it, based on the

        Args:
            params: Parameters to update each step. You don't need to do anythi
                They are properly initialize through the super call.
            lr (float): Learning Rate of the gradient descent.

        Note:
            - YOU ARE NOT ALLOWED to use torch.optim.SGD in this class
            - While you are not allowed to use the class above, it might be ext
            - Make use of pytorch documentation: https://pytorch.org/docs/stable
        """
        super().__init__(params, {"lr": lr})

    @problem.tag("hw3-A")
    def step(self, closure=None):  # noqa: E251
        """
        Performs a step of gradient descent. You should loop through each param

        Args:
            closure (optional): Ignore this. We will not use in this class, but
                Defaults to None.

        Hint:
            - Superclass stores parameters in self.param_groups (you will have
        """
        for group in self.param_groups:
            for param in group['params']:
                if param.grad is not None:
                    param.data.add_(param.grad.data, alpha=-group['lr'])

@problem.tag("hw3-A")
def train(
    train_loader: DataLoader,
    model: nn.Module,
    criterion: nn.Module,
    optimizer: optim.Optimizer,
    val_loader: Optional[DataLoader] = None,
    epochs: int = 100,
) -> Dict[str, List[float]]:
    """Performs training of a provided model and provided dataset.

    Args:
        train_loader (DataLoader): DataLoader for training set.
        model (nn.Module): Model to train.
        criterion (nn.Module): Callable instance of loss function, that can be
        optimizer (optim.Optimizer): Optimizer used for updating parameters of
        val_loader (Optional[DataLoader], optional): DataLoader for validation
            If defined, if should be used to calculate loss on validation set,
            Defaults to None.
        epochs (int, optional): Number of epochs (passes through dataset/datalo
```

```
        Defaults to 100.

    Returns:
        Dict[str, List[float]]: Dictionary with history of training.
            It should have have two keys: "train" and "val",
            each pointing to a list of floats representing loss at each epoch fo
            If val_loader is undefined, "val" can point at an empty list.

    Note:
        - Calculating training loss might expensive if you do it seperately from
            Using a running loss approach is advised.
            In this case you will just use the loss that you called .backward()
            Then you can divide by length of train_loader, and you will have an
        - You will be iterating over multiple models in main function.
            Make sure the optimizer is defined for proper model.
        - Make use of pytorch documentation: https://pytorch.org/docs/stable/ind
            You might find some examples/tutorials useful.
            Also make sure to check out torch.no_grad function. It might be use
        - Make sure to load the model parameters corresponding to model with the
            You might want to look into state_dict: https://pytorch.org/tutoria
    """
    trainLoss = []
    validLoss = []

    for i in range(epochs):
        err = 0.0
        model.train()

        for data, labels in train_loader:
            optimizer.zero_grad()
            target = model(data)
            loss = criterion(target,labels)
            loss.backward()
            err += loss.item()
            optimizer.step()
        trainLoss.append(err / len(train_loader))

        if val_loader == None:
            continue

        model.eval()
        err = 0.0
        with torch.no_grad():
            for data, labels in val_loader:
                target = model(data)
                loss = criterion(target,labels)
                err += loss.item()
            validLoss.append(err / len(val_loader))

    return {"train": trainLoss, "valid": validLoss}


def plot_model_guesses(
    dataloader: DataLoader, model: nn.Module, title: Optional[str] = None
):
```

```
    """Helper function!
    Given data and model plots model predictions, and groups them into:
        - True positives
        - False positives
        - True negatives
        - False negatives

    Args:
        dataloader (DataLoader): Data to plot.
        model (nn.Module): Model to make predictions.
        title (Optional[str], optional): Optional title of the plot.
            Might be useful for distinguishing between MSE and CrossEntropy.
            Defaults to None.
    """
    with torch.no_grad():
        list_xs = []
        list_ys_pred = []
        list_ys_batch = []
        for x_batch, y_batch in dataloader:
            y_pred = model(x_batch)
            list_xs.extend(x_batch.numpy())
            list_ys_batch.extend(y_batch.numpy())
            list_ys_pred.extend(torch.argmax(y_pred, dim=1).numpy())

        xs = np.array(list_xs)
        ys_pred = np.array(list_ys_pred)
        ys_batch = np.array(list_ys_batch)

        # True positive
        if len(ys_batch.shape) == 2 and ys_batch.shape[1] == 2:
            # MSE fix
            ys_batch = np.argmax(ys_batch, axis=1)
        idxs = np.logical_and(ys_batch, ys_pred)
        plt.scatter(
            xs[idxs, 0], xs[idxs, 1], marker="o", c="green", label="True Positiv
        )
        # False positive
        idxs = np.logical_and(1 - ys_batch, ys_pred)
        plt.scatter(
            xs[idxs, 0], xs[idxs, 1], marker="o", c="red", label="False Positive
        )
        # True negative
        idxs = np.logical_and(1 - ys_batch, 1 - ys_pred)
        plt.scatter(
            xs[idxs, 0], xs[idxs, 1], marker="x", c="green", label="True Negativ
        )
        # False negative
        idxs = np.logical_and(ys_batch, 1 - ys_pred)
        plt.scatter(
            xs[idxs, 0], xs[idxs, 1], marker="x", c="red", label="False Negative
        )

        if title:
            plt.title(title)
        plt.xlabel("x0")
```

```
plt.ylabel("x1")
plt.legend()
plt.show()
```

b. *[5 points]* Next we will use the above module to perform hyperparameter search. Here we will also treat loss function as a hyper-parameter. However, because cross-entropy and MSE require different shapes we are going to use two different files: `crossentropy_search.py` and `mean_squared_error_search.py`. For each you will need to build and train (in provided order) 5 models:

- Linear neural network (Single layer, no activation function)
- NN with one hidden layer (2 units) and sigmoid activation function after the hidden layer
- NN with one hidden layer (2 units) and ReLU activation function after the hidden layer
- NN with two hidden layer (each with 2 units) and Sigmoid, ReLU activation functions after first and second hidden layers, respectively
- NN with two hidden layer (each with 2 units) and ReLU, Sigmoid activation functions after first and second hidden layers, respectively

For each loss function, submit a plot of losses from training and validation sets. All models should be on the same plot (10 lines per plot), with two plots total (1 for MSE, 1 for cross-entropy).

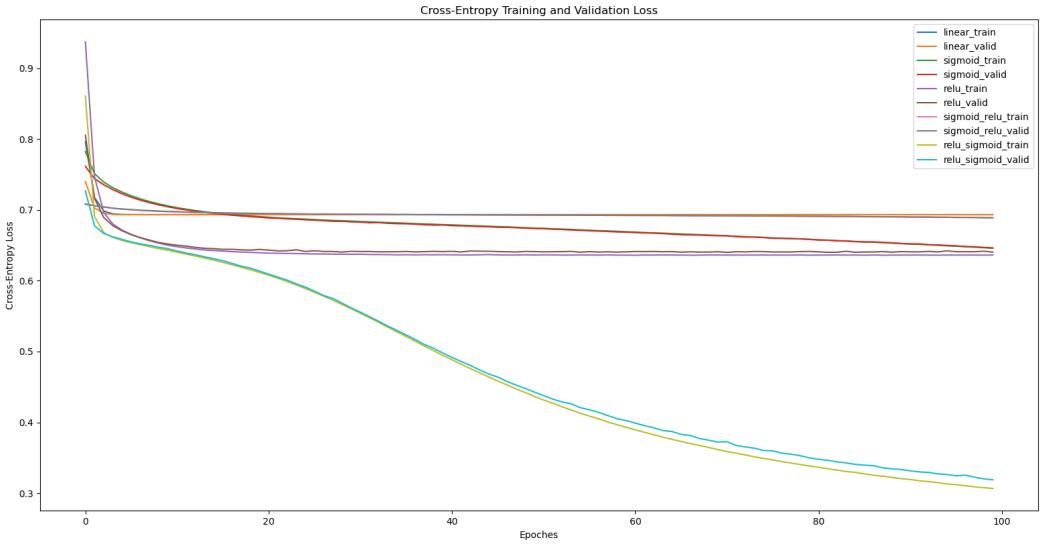


Figure 4: Cross-entropy losses from training and validation sets for various models.

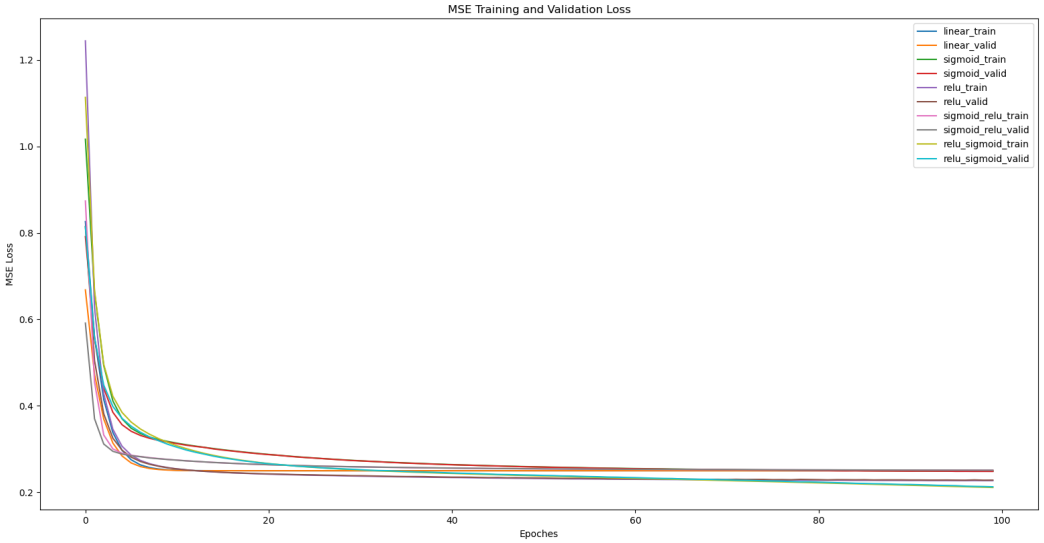


Figure 5: MSE losses from training and validation sets for various models.

```
if __name__ == "__main__":
    from layers import LinearLayer, ReLULayer, SigmoidLayer, SoftmaxLayer
    from losses import CrossEntropyLossLayer
    from optimizers import SGDOptimizer
    from train import plot_model_guesses, train
else:
    from .layers import LinearLayer, ReLULayer, SigmoidLayer, SoftmaxLayer
    from .optimizers import SGDOptimizer
    from .losses import CrossEntropyLossLayer
    from .train import plot_model_guesses, train

from typing import Any, Dict

import numpy as np
import torch
from matplotlib import pyplot as plt
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

from utils import load_dataset, problem

RNG = torch.Generator()
RNG.manual_seed(446)


@problem.tag("hw3-A")
def crossentropy_parameter_search(
    dataset_train: TensorDataset, dataset_val: TensorDataset
) -> Dict[str, Any]:
    """
```

```
Main subroutine of the CrossEntropy problem.
It's goal is to perform a search over hyperparameters, and return a dictionc

Models to check (please try them in this order):
    - Linear Regression Model
    - Network with one hidden layer of size 2 and sigmoid activation functio
    - Network with one hidden layer of size 2 and ReLU activation function
    - Network with two hidden layers (each with size 2)
      and Sigmoid, ReLU activation function after corresponding hidden lay
    - Network with two hidden layers (each with size 2)
      and ReLU, Sigmoid activation function after corresponding hidden lay
NOTE: Each model should end with a Softmax layer due to CrossEntropyLossLay

Notes:
    - Try using learning rate between 1e-5 and 1e-3.
    - When choosing the number of epochs, consider effect of other hyperpara
      For example as learning rate gets smaller you will need more epochs
    - When searching over batch_size using powers of 2 (starting at around
      Make sure it is not too big as you can end up with standard (or alm

Args:
    dataset_train (TensorDataset): Dataset for training.
    dataset_val (TensorDataset): Dataset for validation.

Returns:
    Dict[str, Any]: Dictionary/Map containing history of training of all mo
        You are free to employ any structure of this dictionary, but we sug
        {
            name_of_model: {
                "train": Per epoch losses of model on train set,
                "val": Per epoch losses of model on validation set,
                "model": Actual PyTorch model (type: nn.Module),
            }
        }
"""
trainLoader = DataLoader(dataset_train, batch_size=32, shuffle=True)
validLoader = DataLoader(dataset_val, batch_size=32, shuffle=True)
learnRate = 0.005

output = dict()
criterion = CrossEntropyLossLayer()

linear = nn.Sequential(
    LinearLayer(2, 2, generator=RNG),
    SoftmaxLayer()
)
opt1 = SGDOptimizer(params=linear.parameters(), lr=learnRate)
trainRes1 = train(trainLoader, linear, criterion, opt1, validLoader, 100)
output['linear'] = {"train": trainRes1["train"], "valid": trainRes1["valid"]

sigmoid = nn.Sequential(
    LinearLayer(2, 2, generator=RNG),
    SigmoidLayer(),
    LinearLayer(2, 2, generator=RNG),
    SoftmaxLayer()
```

```
    )
    opt2 = SGDOptimizer(params=sigmoid.parameters(), lr=learnRate)
    trainRes2 = train(trainLoader, sigmoid, criterion, opt2, validLoader, 100)
    output['sigmoid'] = {"train": trainRes2["train"], "valid": trainRes2["valid"

    relu = nn.Sequential(
        LinearLayer(2, 2, generator=RNG),
        ReLULayer(),
        LinearLayer(2, 2, generator=RNG),
        SoftmaxLayer()
    )
    opt3 = SGDOptimizer(params=relu.parameters(), lr=learnRate)
    trainRes3 = train(trainLoader, relu, criterion, opt3, validLoader, 100)
    output['relu'] = {"train": trainRes3["train"], "valid": trainRes3["valid"],

    sigmoid_relu = nn.Sequential(
        LinearLayer(2, 2, generator=RNG),
        SigmoidLayer(),
        LinearLayer(2, 2, generator=RNG),
        ReLULayer(),
        LinearLayer(2, 2, generator=RNG),
        SoftmaxLayer()
    )
    opt4 = SGDOptimizer(params=sigmoid_relu.parameters(), lr=learnRate)
    trainRes4 = train(trainLoader, sigmoid_relu, criterion, opt4, validLoader, 
    output['sigmoid_relu'] = {"train": trainRes4["train"], "valid": trainRes4["

    relu_sigmoid = nn.Sequential(
        LinearLayer(2, 2, generator=RNG),
        ReLULayer(),
        LinearLayer(2, 2, generator=RNG),
        SigmoidLayer(),
        LinearLayer(2, 2, generator=RNG),
        SoftmaxLayer()
    )
    opt5 = SGDOptimizer(params=relu_sigmoid.parameters(), lr=learnRate)
    trainRes5 = train(trainLoader, relu_sigmoid, criterion, opt5, validLoader, 
    output['relu_sigmoid'] = {"train": trainRes5["train"], "valid": trainRes5["

    return output


@problem.tag("hw3-A")
def accuracy_score(model, dataloader) -> float:
    """Calculates accuracy of model on dataloader. Returns it as a fraction.

    Args:
        model (nn.Module): Model to evaluate.
        dataloader (DataLoader): Dataloader for CrossEntropy.
            Each example is a tuple consiting of (observation, target).
            Observation is a 2-d vector of floats.
            Target is an integer representing a correct class to a correspondin

    Returns:
        float: Vanilla python float resprenting accuracy of the model on given 
```

```
            In range [0, 1].

    Note:
        - For a single-element tensor you can use .item() to cast it to a float
        - This is similar to MSE accuracy_score function,
            but there will be differences due to slightly different targets in
    """
    corr = 0; tot = 0
    with torch.no_grad():
        for data in dataloader:
            obs, tar = data
            out = model(obs)
            _, pred = torch.max(out.data, 1)
            tot += tar.size(0)
            corr += (pred == tar).sum().item()
    return corr / tot


@problem.tag("hw3-A", start_line=7)
def main():
    """
    Main function of the Crossentropy problem.
    It should:
        1. Call crossentropy_parameter_search routine and get dictionary for ea
        2. Plot Train and Validation losses for each model all on single plot (
            x-axis should be epochs, y-axis should me Crossentropy loss, REMEMB
        3. Choose and report the best model configuration based on validation lo
            In particular you should choose a model that achieved the lowest va
        4. Plot best model guesses on test set (using plot_model_guesses functio
        5. Report accuracy of the model on test set.

    Starter code loads dataset, converts it into PyTorch Datasets, and those in
    You should use these dataloaders, for the best experience with PyTorch.
    """
    (x, y), (x_val, y_val), (x_test, y_test) = load_dataset("xor")

    dataset_train = TensorDataset(torch.from_numpy(x), torch.from_numpy(y))
    dataset_val = TensorDataset(torch.from_numpy(x_val), torch.from_numpy(y_val)
    dataset_test = TensorDataset(torch.from_numpy(x_test), torch.from_numpy(y_t

    ce_configs = crossentropy_parameter_search(dataset_train, dataset_val)
    minLoss = float("inf")
    minModel = None
    model = None

    for i in ce_configs.items():
        x = range(100)
        train = i[1]['train']
        valid = i[1]['valid']
        currModel = i[0]
        mod = i[1]["model"]
        plt.plot(x , train, label = currModel + "_train")
        plt.plot(x , valid, label = currModel + "_valid")
        currLoss = min(valid)
        if currLoss < minLoss:
```

```
                minLoss = currLoss
                minModel = currModel
                model = mod

        plt.ylabel("Cross-Entropy Loss")
        plt.xlabel("Epoches")
        plt.title("Cross-Entropy Training and Validation Loss")
        plt.legend()
        plt.show()

        print("Best performing architecture: " + minModel)
        plot_model_guesses(DataLoader(dataset_test), model, minModel)
        acc = accuracy_score(model, DataLoader(dataset_test))
        print(acc)

if __name__ == "__main__":
    from layers import LinearLayer, ReLULayer, SigmoidLayer
    from losses import MSELossLayer
    from optimizers import SGDOptimizer
    from train import plot_model_guesses, train
else:
    from .layers import LinearLayer, ReLULayer, SigmoidLayer
    from .optimizers import SGDOptimizer
    from .losses import MSELossLayer
    from .train import plot_model_guesses, train

from typing import Any, Dict

import numpy as np
import torch
from matplotlib import pyplot as plt
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

from utils import load_dataset, problem

RNG = torch.Generator()
RNG.manual_seed(446)


@problem.tag("hw3-A")
def accuracy_score(model: nn.Module, dataloader: DataLoader) -> float:
    """Calculates accuracy of model on dataloader. Returns it as a fraction.

    Args:
        model (nn.Module): Model to evaluate.
        dataloader (DataLoader): Dataloader for MSE.
            Each example is a tuple consiting of (observation, target).
            Observation is a 2-d vector of floats.
            Target is also a 2-d vector of floats, but specifically with one be
            Index of 1.0 in target corresponds to the true class.

    Returns:
        float: Vanilla python float resprenting accuracy of the model on given
            In range [0, 1].
```

```
    Note:
        - For a single-element tensor you can use .item() to cast it to a float
        - This is similar to CrossEntropy accuracy_score function,
            but there will be differences due to slightly different targets in
    """
    corr = 0
    tot = 0
    with torch.no_grad():
        for data in dataloader:
            obs, tar = data
            out = model(obs)
            _, pred = torch.max(out.data, 1)
            _, tar = torch.max(tar.data, 1)
            tot += tar.size(0)
            corr += (pred == tar).sum().item()
    return corr / tot


@problem.tag("hw3-A")
def mse_parameter_search(
    dataset_train: TensorDataset, dataset_val: TensorDataset
) -> Dict[str, Any]:
    """
    Main subroutine of the MSE problem.
    It's goal is to perform a search over hyperparameters, and return a dictiona

    Models to check (please try them in this order):
        - Linear Regression Model
        - Network with one hidden layer of size 2 and sigmoid activation functio
        - Network with one hidden layer of size 2 and ReLU activation function
        - Network with two hidden layers (each with size 2)
            and Sigmoid, ReLU activation function after corresponding hidden lay
        - Network with two hidden layers (each with size 2)
            and ReLU, Sigmoid activation function after corresponding hidden lay

    Notes:
        - Try using learning rate between 1e-5 and 1e-3.
        - When choosing the number of epochs, consider effect of other hyperpara
            For example as learning rate gets smaller you will need more epochs
        - When searching over batch_size using powers of 2 (starting at around
            Make sure it is not too big as you can end up with standard (or alm

    Args:
        dataset_train (TensorDataset): Training dataset.
        dataset_val (TensorDataset): Validation dataset.

    Returns:
        Dict[str, Any]: Dictionary/Map containing history of training of all mo
            You are free to employ any structure of this dictionary, but we sug
            {
                name_of_model: {
                    "train": Per epoch losses of model on train set,
                    "val": Per epoch losses of model on validation set,
                    "model": Actual PyTorch model (type: nn.Module),
```

```
            }
        }
"""
trainLoader = DataLoader(dataset_train, batch_size=32, shuffle=True)
validLoader = DataLoader(dataset_val, batch_size=32, shuffle=True)

output = dict()
criterion = MSELossLayer()
learnRate = 0.001

linear = nn.Sequential(
    LinearLayer(2, 2, generator=RNG)
)
opt1 = SGDOptimizer(params=linear.parameters(), lr=learnRate)
trainRes1 = train(trainLoader, linear, criterion, opt1, validLoader, 100)
output['linear'] = { "train": trainRes1["train"], "valid": trainRes1["valid"

sigmoid = nn.Sequential(
    LinearLayer(2, 2, generator=RNG),
    SigmoidLayer(),
    LinearLayer(2, 2, generator=RNG)
)
opt2 = SGDOptimizer(params=sigmoid.parameters(), lr=learnRate)
trainRes2 = train(trainLoader, sigmoid, criterion, opt2, validLoader, 100)
output['sigmoid'] = { "train": trainRes2["train"], "valid": trainRes2["valid

relu = nn.Sequential(
    LinearLayer(2, 2, generator=RNG),
    ReLULayer(),
    LinearLayer(2, 2, generator=RNG)
)
opt3 = SGDOptimizer(params=relu.parameters(), lr=learnRate)
trainRes3 = train(trainLoader, relu, criterion, opt3, validLoader, 100)
output['relu'] = { "train": trainRes3["train"], "valid": trainRes3["valid"]

sigmoid_relu = nn.Sequential(
    LinearLayer(2, 2, generator=RNG),
    SigmoidLayer(),
    LinearLayer(2, 2, generator=RNG),
    ReLULayer(),
    LinearLayer(2, 2, generator=RNG)
)
opt4 = SGDOptimizer(params=sigmoid_relu.parameters(), lr=learnRate)
trainRes4 = train(trainLoader, sigmoid_relu, criterion, opt4, validLoader, 
output['sigmoid_relu'] = { "train": trainRes4["train"], "valid": trainRes4["

relu_sigmoid = nn.Sequential(
    LinearLayer(2, 2, generator=RNG),
    ReLULayer(),
    LinearLayer(2, 2, generator=RNG),
    SigmoidLayer(),
    LinearLayer(2, 2, generator=RNG)
)
opt5 = SGDOptimizer(params=relu_sigmoid.parameters(), lr=learnRate)
trainRes5 = train(trainLoader, relu_sigmoid, criterion, opt5, validLoader, 
```

```
    output['relu_sigmoid'] = { "train": trainRes5["train"], "valid": trainRes5['

    return output


@problem.tag("hw3-A", start_line=11)
def main():
    """
    Main function of the MSE problem.
    It should:
        1. Call mse_parameter_search routine and get dictionary for each model
        2. Plot Train and Validation losses for each model all on single plot (
            x-axis should be epochs, y-axis should me MSE loss, REMEMBER to add
        3. Choose and report the best model configuration based on validation lo
            In particular you should choose a model that achieved the lowest va
        4. Plot best model guesses on test set (using plot_model_guesses functio
        5. Report accuracy of the model on test set.

    Starter code loads dataset, converts it into PyTorch Datasets, and those int
    You should use these dataloaders, for the best experience with PyTorch.
    """
    (x, y), (x_val, y_val), (x_test, y_test) = load_dataset("xor")

    dataset_train = TensorDataset(torch.from_numpy(x), torch.from_numpy(to_one_h
    dataset_val = TensorDataset(
        torch.from_numpy(x_val), torch.from_numpy(to_one_hot(y_val))
    )
    dataset_test = TensorDataset(
        torch.from_numpy(x_test), torch.from_numpy(to_one_hot(y_test))
    )

    mse_configs = mse_parameter_search(dataset_train, dataset_val)
    minLoss = float("inf")
    minModel = None
    model = None

    for i in mse_configs.items():
        x = range(100)
        train = i[1]['train']
        valid = i[1]['valid']
        currModel = i[0]
        mod = i[1]["model"]
        plt.plot(x , train, label = currModel + "_train")
        plt.plot(x , valid, label = currModel + "_valid")
        currLoss = min(valid)
        if currLoss < minLoss:
            minLoss = currLoss
            minModel = currModel
            model = mod


    plt.ylabel("MSE_Loss")
    plt.xlabel("Epoches")
    plt.title("MSE_Training_and_Validation_Loss")
    plt.legend()
```

```
    plt.show()

    print("Best performing architecture: " + minModel)

    plot_model_guesses(DataLoader(dataset_test), model, minModel)
    acc = accuracy_score(model, DataLoader(dataset_test))
    print(acc)


def to_one_hot(a: np.ndarray) -> np.ndarray:
    """Helper function. Converts data from categorical to one-hot encoded.

    Args:
        a (np.ndarray): Input array of integers with shape (n,).

    Returns:
        np.ndarray: Array with shape (n, c), where c is maximal element of a.
            Each element of a, has a corresponding one-hot encoded vector of le
    """
    r = np.zeros((len(a), 2))
    r[np.arange(len(a)), a] = 1
    return r
```

c. *[5 points]* For each loss function, report the best performing architecture (best performing is defined here as achieving the lowest validation loss at any point during the training), and plot it's guesses on test set. You should use function `plot_model_guesses` from `train.py` file. Lastly, report accuracy of that model on a test set.

Cross Entropy Loss: ReLU-sigmoid (accuracy: 0.907)

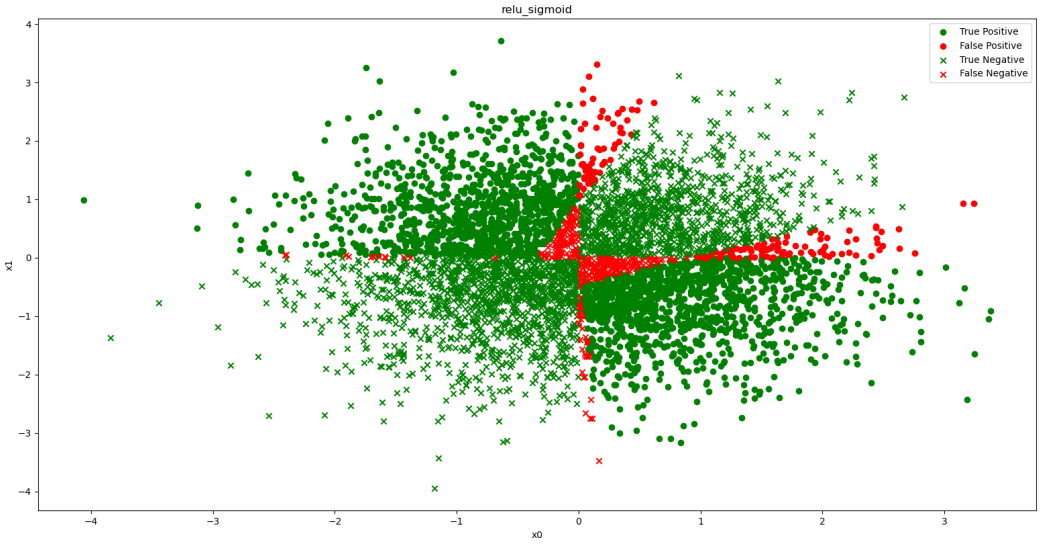


Figure 6: Scatter plot of ReLU-sigmoid model predictions on test set with cross-entropy loss function.

MSE Loss: ReLU-sigmoid (accuracy: 0.657)

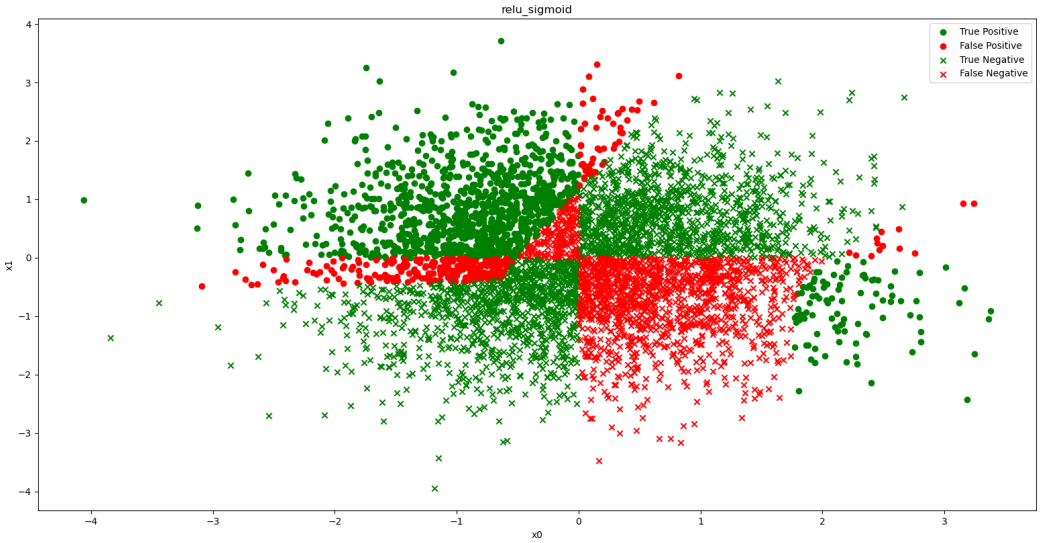


Figure 7: Scatter plot of ReLU-sigmoid model predictions on test set with MSE loss function.

On Softmax function

One of the activation functions we ask you to implement is softmax. For a prediction $\hat{y} \in \mathbb{R}^k$ corresponding to single datapoint (in a problem with k classes):

$$\text{softmax}(\hat{y}_i) = \frac{\exp(\hat{y}_i)}{\sum_j \exp(\hat{y}_j)}$$

Neural Networks for MNIST

Resources

For questions A.4, A.5 and A.6 you will use a lot of PyTorch. In Section materials (Week 6) there is a notebook that you might find useful. Additionally make use of PyTorch Documentation, when needed.
If you do not have access to GPU, you might find Google Colaboratory useful. It allows you to use a cloud GPU for free. To enable it make sure: "Runtime" -¿ "Change runtime type" -¿ "Hardware accelerator" is set to "GPU". When submitting please download and submit a `.py` version of your notebook.

A6. In Homework 1, we used ridge regression for training a classifier for the MNIST data set. In Homework 2, we used logistic regression to distinguish between the digits 2 and 7. In this problem, we will use PyTorch to build a simple neural network classifier for MNIST to further improve our accuracy.

We will implement two different architectures: a shallow but wide network, and a narrow but deeper network. For both architectures, we use d to refer to the number of input features (in MNIST, $d = 28^2 = 784$), h_i to refer to the dimension of the i-th hidden layer and k for the number of target classes (in MNIST, $k = 10$). For the non-linear activation, use ReLU. Recall from lecture that

$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}.$$

Weight Initialization

Consider a weight matrix $W \in \mathbb{R}^{n \times m}$ and $b \in \mathbb{R}^n$. Note that here m refers to the input dimension and n to the output dimension of the transformation $x \mapsto Wx + b$. Define $\alpha = \frac{1}{\sqrt{m}}$. Initialize all your weight matrices and biases according to $\text{Unif}(-\alpha, \alpha)$.

Training

For this assignment, use the Adam optimizer from `torch.optim`. Adam is a more advanced form of gradient descent that combines momentum and learning rate scaling. It often converges faster than regular gradient descent in practice. You can use either Gradient Descent or any form of Stochastic Gradient Descent. Note that you are still using Adam, but might pass either the full data, a single datapoint or a batch of data to it. Use cross entropy for the loss function and ReLU for the non-linearity.

Implementing the Neural Networks

a. *[10 points]* Let $W_0 \in \mathbb{R}^{h \times d}$, $b_0 \in \mathbb{R}^h$, $W_1 \in \mathbb{R}^{k \times h}$, $b_1 \in \mathbb{R}^k$ and $\sigma(z) \colon \mathbb{R} \to \mathbb{R}$ some non-linear activation function applied element-wise. Given some $x \in \mathbb{R}^d$, the forward pass of the wide, shallow network can be formulated as:

$$\mathcal{F}_1(x) \coloneqq W_1 \sigma(W_0 x + b_0) + b_1$$

Use $h = 64$ for the number of hidden units and choose an appropriate learning rate. Train the network until it reaches 99% accuracy on the training data and provide a training plot (loss vs. epoch). Finally evaluate the model on the test data and report both the accuracy and the loss.

$\mathcal{F}_1$ model test loss is 0.10442 and test accuracy is 0.967.

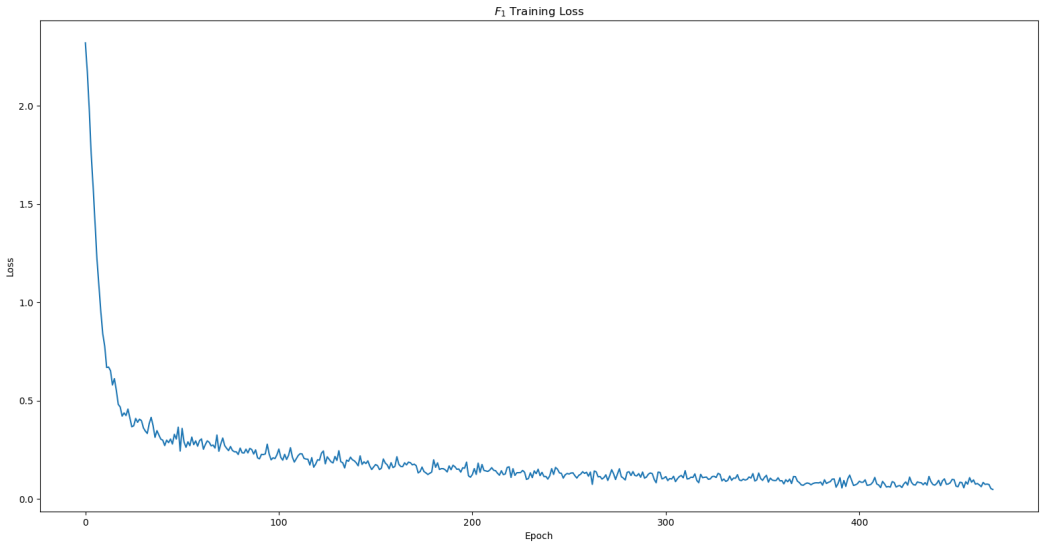


Figure 8: $\mathcal{F}_1$ training loss vs. epoch.

b. *[10 points]* Let $W_0 \in \mathbb{R}^{h_0 \times d}$, $b_0 \in \mathbb{R}^{h_0}$, $W_1 \in \mathbb{R}^{h_1 \times h_0}$, $b_1 \in \mathbb{R}^{h_1}$, $W_2 \in \mathbb{R}^{k \times h_1}$, $b_2 \in \mathbb{R}^k$ and $\sigma(z) : \mathbb{R} \to \mathbb{R}$ some non-linear activation function. Given some $x \in \mathbb{R}^d$, the forward pass of the network can be formulated as:

$$\mathcal{F}_2(x) \coloneqq W_2\sigma(W_1\sigma(W_0x + b_0) + b_1) + b_2$$

Use $h_0 = h_1 = 32$ and perform the same steps as in part a.

$\mathcal{F}_2$ model test loss is 0.12276 and test accuracy is 0.964.

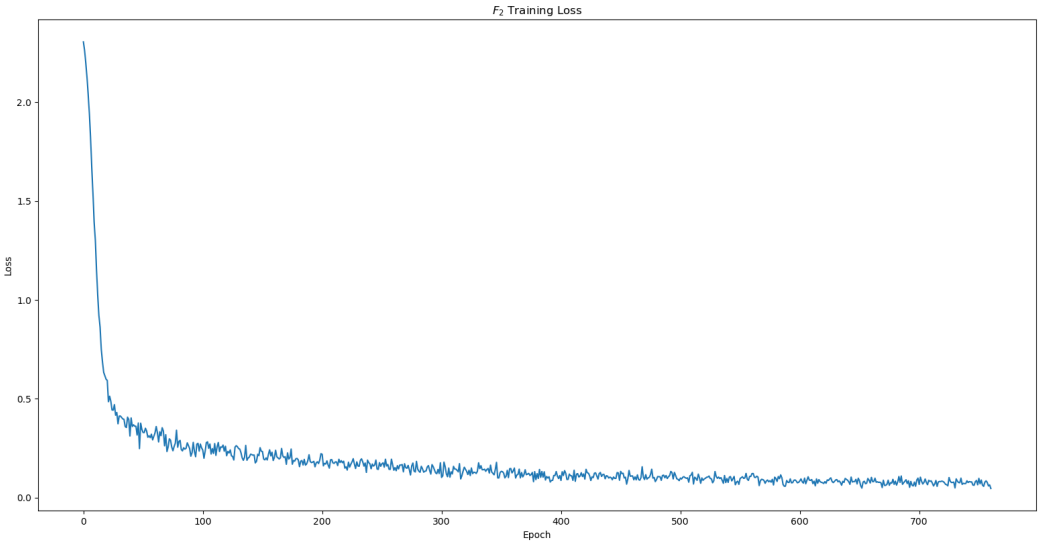


Figure 9: $\mathcal{F}_2$ training loss vs. epoch.

c. *[5 points]* Compute the total number of parameters of each network and report them. Then compare the number of parameters as well as the test accuracies the networks achieved. Is one of the approaches (wide, shallow vs. narrow, deeper) better than the other? Give an intuition for why or why not.

The total number of parameters in the $\mathcal{F}_1$ model is 50890.
The total number of parameters in the $\mathcal{F}_2$ model is 26506.
Both models output similar results when we compare the test evaluations. The three-layer model is faster because there are fewer parameters.

Using PyTorch: For your solution, you may not use any functionality from the `torch.nn` module except for `torch.nn.functional.relu` and `torch.nn.functional.cross_entropy`. You must implement the networks $\mathcal{F}_1$ and $\mathcal{F}_2$ from scratch. For starter code and a tutorial on PyTorch refer to the sections 6 and 7 material.

```
class F1(Module):
    @problem.tag("hw3-A", start_line=1)
    def __init__(self, h: int, d: int, k: int):
        """Create a F1 model as described in pdf.

        Args:
            h (int): Hidden dimension.
            d (int): Input dimension/number of features.
            k (int): Output dimension/number of classes.
        """
```

```
        super().__init__()
        self.weight_0 = Parameter(Uniform(-1 / math.sqrt(d), 1 / math.sqrt(d)).sample((h
        self.bias_0 = Parameter(Uniform(-1 / math.sqrt(d), 1 / math.sqrt(d)).sample((h,)
        self.weight_1 = Parameter(Uniform(-1 / math.sqrt(h), 1 / math.sqrt(h)).sample((k
        self.bias_1 = Parameter(Uniform(-1 / math.sqrt(h), 1 / math.sqrt(h)).sample((k,)

    @problem.tag("hw3-A")
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Pass input through F1 model.

        It should perform operation:
        W_1(sigma(W_0*x + b_0)) + b_1

        Note that in this coding assignment, we use the same convention as previous
        assignments where a linear module is of the form xW + b. This differs from the
        general forward pass operation defined above, which assumes the form Wx + b.
        When implementing the forward pass, make sure that the correct matrices and
        transpositions are used.

        Args:
            x (torch.Tensor): FloatTensor of shape (n, d). Input data.

        Returns:
            torch.Tensor: FloatTensor of shape (n, k). Prediction.
        """
        hidden_layer = torch.matmul(x, torch.transpose(self.weight_0, 0, 1)) + self.bias
        hidden_layer_relu = relu(hidden_layer)
        return torch.matmul(hidden_layer_relu, torch.transpose(self.weight_1, 0, 1)) + s


class F2(Module):
    @problem.tag("hw3-A", start_line=1)
    def __init__(self, h0: int, h1: int, d: int, k: int):
        """Create a F2 model as described in pdf.

        Args:
            h0 (int): First hidden dimension (between first and second layer).
            h1 (int): Second hidden dimension (between second and third layer).
            d (int): Input dimension/number of features.
            k (int): Output dimension/number of classes.
        """
        super().__init__()
        self.weight_0 = Parameter(Uniform(-1 / math.sqrt(d), 1 / math.sqrt(d)).sample((h
        self.bias_0 = Parameter(Uniform(-1 / math.sqrt(d), 1 / math.sqrt(d)).sample((h0,

        self.weight_1 = Parameter(Uniform(-1 / math.sqrt(h0), 1 / math.sqrt(h0)).sample(
        self.bias_1 = Parameter(Uniform(-1 / math.sqrt(h0), 1 / math.sqrt(h0)).sample((h

        self.weight_2 = Parameter(Uniform(-1 / math.sqrt(h1), 1 / math.sqrt(h1)).sample(
        self.bias_2 = Parameter(Uniform(-1 / math.sqrt(h1), 1 / math.sqrt(h1)).sample((k

    @problem.tag("hw3-A")
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
```

```
        Pass input through F2 model.

        It should perform operation:
        W_2(sigma(W_1(sigma(W_0*x + b_0)) + b_1) + b_2)

        Note that in this coding assignment, we use the same convention as previous
        assignments where a linear module is of the form xW + b. This differs from the
        general forward pass operation defined above, which assumes the form Wx + b.
        When implementing the forward pass, make sure that the correct matrices and
        transpositions are used.

        Args:
            x (torch.Tensor): FloatTensor of shape (n, d). Input data.

        Returns:
            torch.Tensor: FloatTensor of shape (n, k). Prediction.
        """
        hidden_layer_0 = torch.matmul(x, torch.transpose(self.weight_0, 0, 1)) + self.bi
        hidden_layer_0_relu = relu(hidden_layer_0)
        hidden_layer_1 = torch.matmul(hidden_layer_0_relu, torch.transpose(self.weight_1
        hidden_layer_1_relu = relu(hidden_layer_1)
        return torch.matmul(hidden_layer_1_relu, torch.transpose(self.weight_2, 0, 1)) +


@problem.tag("hw3-A")
def train(model: Module, optimizer: Adam, train_loader: DataLoader) -> List[float]:
    """
    Train a model until it reaches 99% accuracy on train set, and return list of trainin

    Args:
        model (Module): Model to train. Either F1, or F2 in this problem.
        optimizer (Adam): Optimizer that will adjust parameters of the model.
        train_loader (DataLoader): DataLoader with training data.
            You can iterate over it like a list, and it will produce tuples (x, y),
            where x is FloatTensor of shape (n, d) and y is LongTensor of shape (n,).
            Note that y contains the classes as integers.

    Returns:
        List[float]: List containing average loss for each epoch.
    """
    optimizer = Adam(model.parameters(), lr=5e-3)
    loss = []
    accuracy = 0

    while accuracy < 0.99:
        x, y = next(iter(train_loader))
        optimizer.zero_grad()
        y_pred = model(x)

        loss_in_epoch = cross_entropy(y_pred, y)
        loss_in_epoch.backward()
        optimizer.step()

        loss.append(loss_in_epoch.item())
        accuracy = torch.sum(torch.argmax(y_pred, dim=1) == y).item()/len(y)
```

```
    return loss


@problem.tag("hw3-A", start_line=5)
def main():
    """
    Main function of this problem.
    For both F1 and F2 models it should:
        1. Train a model
        2. Plot per epoch losses
        3. Report accuracy and loss on test set
        4. Report total number of parameters for each network

    Note that we provided you with code that loads MNIST and changes x's and y's to cor
    We strongly advise that you use torch functionality such as datasets, but as mentio
    """
    (x, y), (x_test, y_test) = load_dataset("mnist")
    x = torch.from_numpy(x).float()
    y = torch.from_numpy(y).long()
    x_test = torch.from_numpy(x_test).float()
    y_test = torch.from_numpy(y_test).long()

    train_dataset = TensorDataset(x, y)
    train_loader = DataLoader(train_dataset, batch_size=1000, shuffle=True)

    f1_model = F1(h=64, d=784, k=10)
    f1_model_loss = train(f1_model, Adam, train_loader)

    fig, ax = plt.subplots()
    ax.plot(f1_model_loss)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.set_title('$F_1$ Training Loss')
    plt.show()

    y_test_pred_f1 = f1_model(x_test)
    f1_test_loss = cross_entropy(y_test_pred_f1, y_test).item()
    f1_test_accuracy = torch.sum(torch.argmax(y_test_pred_f1, dim=1) == y_test).item()/l
    print(f'F1 model test loss is {f1_test_loss} and test accuracy is {f1_test_accuracy}

    f2_model = F2(h0=32, h1=32, d=784, k=10)
    f2_model_loss = train(f2_model, Adam, train_loader)

    fig, ax = plt.subplots()
    ax.plot(f2_model_loss)
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.set_title('$F_2$ Training Loss')
    plt.show()

    y_test_pred_f2 = f2_model(x_test)
    f2_test_loss = cross_entropy(y_test_pred_f2, y_test).item()
    f2_test_accuracy = torch.sum(torch.argmax(y_test_pred_f2, dim=1) == y_test).item()/l
    print(f'F2 model test loss is {f2_test_loss} and test accuracy is {f2_test_accuracy}
```

```
k = 10
d = 784
h = 64
f1_total_params = ((h * d) + h) + ((k*h) + k)
print(f'Total number of parameters in F1 model is {f1_total_params}')
h0 = 32
h1 = 32
f2_total_params = ((h0*d) + h0) + ((h1*h0) + h1) + ((k*h1) + k)
print(f'Total number of parameters in F2 model is {f2_total_params}')
```